Rotaract Club of Xiamen University Malaysia | Rotary Club of Kuala Lumpur Dynamics | Rotary Club of Tun Razak Exchange

SDC Hackathon

Technical Tools Declaration

Hosted by Xiamen University Malaysia (XMUM) AI Club

Submission Deadline: 25 October 2026

Instructions: Declare every API, cloud service, AI tool, dataset, model, and sponsor resource your team used to build the project. Complete every section; where a category does not apply, tick "No" or write "None" – do not leave it blank. Submit as a single PDF document named "SDC2026_TeamName_TechDeclaration.pdf" together with your project submission. Undeclared third-party resources discovered at judging may affect your score or eligibility.

Instagram: @sdc_xmumhack ***Email:*** sdc.xmumhack@gmail.com

SDC HACKATHON 2026

Technical & Tools Declaration

Hosted by Xiamen University Malaysia (XMUM) AI Club

Submission Deadline: [Insert date]

Instructions: Declare every API, cloud service, AI tool, dataset, model, and sponsor resource your team used to build the project. Complete every section; where a category does not apply, tick "No" or write "None" – do not leave it blank. Submit as a single PDF document named "SDC2026_TeamName_TechDeclaration.pdf" together with your project submission. Undeclared third-party resources discovered at judging may affect your score or eligibility.

1. Team & Project Overview

Team Name	Steady Bom Bi Bi
Project / Solution Name	BeliKongsi
Name (Leader)	Shawn Lee Yen Zheng
Email & Phone (Leader)	shawnlee0206@gmail.com (010-2989939)
University Affiliation(s)	Asia Pacific University, Sunway University
Live Prototype URL	https://beli-kongsi.replit.app
Code Repository URL (public / access granted to judges)	https://github.com/ShawnLYZ/BeliKongsi

2. Architecture & Technology Stack

2.1 System Architecture Overview

Describe how the components declared in this document connect end-to-end (e.g., user interface → backend → AI model/API → database → cloud hosting). Attach a system block diagram as an appendix if available.

[Maximum 200 words]

BeliKongsi is built on an end-to-end cloud-native architecture connecting five core tiers:

1. **User Interface:** A mobile-first Next.js (React 19, Tailwind CSS) web client provides role-specific portals (Shop Owners, Staff, Suppliers, and Ops) with Google Maps for hub visualization.
2. **Backend & Security:** Next.js App Router server routes validate inputs with Zod, authenticate sessions, and enforce server-side role checks.
3. **Core Engine:** The pure, zero-dependency **Kongsi Engine** calculates order pooling, wholesale tier thresholds, delivery splits, and automated ledger refunds deterministically.
4. **AI Layer:** Server-side adapters connect to **Google Gemini** via the @google/genai SDK to convert unstructured WhatsApp restock messages into structured draft orders. AI outputs are advisory only and require human-in-the-loop approval.
5. **Database & Storage:** Persisted in **PostgreSQL** (hosted on **Supabase**) accessed exclusively server-side via **Prisma ORM** under a restricted role with Row-Level Security (RLS). Receipts and proof attachments reside in Supabase Storage.
6. **Hosting:** Deployed on **Vercel** serverless infrastructure, orchestrating secure TLS communication across external APIs.

2.2 Languages, Frameworks & Libraries

List the main programming languages, frameworks, and third-party libraries that your team did not create. Open-source use is permitted provided it is declared and properly licensed/attributed.

Name & Version	Purpose in Project	License
TypeScript (v6.0.3)	Primary programming language providing strict static typing for frontend, backend services, and the deterministic Kongsi Engine.	Apache 2.0
Next.js (v16.4.2)	Core full-stack framework powering the App Router, server-side API route handlers, and serverless deployment on Vercel.	MIT
React & React DOM (v19.3.1)	Declarative UI library for building reactive, mobile-first web interfaces for shop owners, staff, suppliers, and ops.	MIT
Tailwind CSS (v4.3.3)	Utility-first CSS framework for responsive layout design, design tokens, and UI component styling.	MIT
Prisma ORM (v7.10.1)	Type-safe database ORM and migration engine for interacting with the PostgreSQL database.	Apache 2.0
@google/genai (v2.30.0)	Official Google Gemini SDK used server-side to parse unstructured WhatsApp restock messages into structured draft order lines.	Apache 2.0
@supabase/supabase-js (v2.118.0)	Cloud integration client for connecting to managed PostgreSQL database services and cloud object storage for receipts.	MIT
@vis.gl/react-google-maps (v1.10.1)	React component wrapper for Google Maps API to visualize regional supplier hubs and pickup collection points.	MIT
Zod (v4.6.7)	Schema declaration and validation library for validating runtime API request payloads and environment variables.	MIT
Radix UI (v1.8.1) & Lucide Icons (v1.60.0)	Accessible unstyled UI component primitives and iconography for mobile menus, modals, and buttons.	MIT / ISC

Name & Version	Purpose in Project	License
Sharp (v0.35.5)	High-performance image processing library used for resizing and formatting product catalog imagery.	Apache 2.0
node-postgres / pg (v8.23.1)	PostgreSQL client driver providing connection pooling for server-side database communication.	MIT

3. APIs & Cloud Services

3.1 External APIs

List every external API your solution calls (GenAI, maps, payments, messaging, data feeds, etc.). Do not include API keys or secrets in this document.

API & Provider	Purpose in Project	Access Tier (free / paid / sponsor-provided)
Google Gemini API *(Provider: Google Cloud / Google AI Studio)*	Natural language parsing of raw, unstructured WhatsApp restock messages into structured order line items and confidence scores.	Free tier / Pay-as-you-go
Google Maps JavaScript API *(Provider: Google Cloud Platform)*	Interactive map rendering for regional consolidation hubs, supplier drop-off points, and shop pickup collection locations.	Free tier *(within Google Cloud's monthly recurring credit)*
Supabase Cloud & Storage API *(Provider: Supabase Inc.)*	Server-side cloud database connection pooling (PostgreSQL) and object storage bucket for purchase receipts and delivery photos.	Free tier *(Supabase Free Tier)*
NVIDIA NIM API — FLUX.2 Klein *(Provider: NVIDIA)*	Offline generative AI endpoint used in development scripts (scripts/generate-product-images.ts) to generate realistic product catalogue images.	Free trial *(Developer API credits)*

3.2 Cloud Platforms & Deployment

List every cloud service used for hosting, storage, databases, authentication, or compute, and indicate how the prototype is deployed.

Service & Provider	**Purpose in Project**	**Plan / Region**
Vercel	Hosting & Serverless Compute	Hosts the Next.js full-stack application. Deployed in region sin1 (Singapore) for low-latency co-location beside the database. Executes serverless route handlers, background cron jobs, and edge asset delivery.
Vercel Cron	Scheduled Jobs	Triggers automated weekly restock drafting every Monday at 06:00 MYT (/api/jobs/run) secured by CRON_SECRET.
Supabase *(AWS ap-southeast-1, Singapore)*	Managed Database (PostgreSQL)	Persistent transactional storage using managed PostgreSQL with Supavisor connection pooling (transaction mode on port 6543). Enforces Row-Level Security (RLS) and restricted database role privileges (bk_app).
Supabase Storage	Cloud Object Storage	S3-compatible cloud bucket for storing and serving uploaded restock receipts, supplier invoices, and delivery check photos.

Service & Provider	Purpose in Project	Plan / Region
Server-Side Session Auth	Authentication	Cryptographically signed, stateless session cookies (HMAC-SHA256 signed with SESSION_SECRET), enforcing server-side Role-Based Access Control (RBAC) across 4 personas: Shop Owner, Staff, Supplier, and Ops.
Google Cloud Platform	External AI & Maps	Provides Google Gemini API endpoints for natural language order reading and Google Maps Platform for distribution hub visualization.

4. AI Tools, Models & Datasets

4.1 AI Models Used in the Solution

List every AI model embedded in or called by your application, including pre-trained, fine-tuned, and self-trained models.

Model & Version	Source / Provider	Usage (pre-trained / fine-tuned / self-trained)	License / Terms
Gemini 3.5 Flash Lite *(Configurable fallback: Gemini 2.5 Flash)*	Google Cloud / Google AI Studio	Pre-trained *(Prompt-grounded with JSON Schema output constraints; no fine-tuning)*	Google APIs Terms of Service / Google AI Terms of Use
FLUX.2-klein-4b	Black Forest Labs *(accessed via NVIDIA NIM API)*	Pre-trained *(Offline generation script for synthetic demo product catalogue images)*	FLUX.2 Klein Non-Commercial / NVIDIA AI Foundation Models Terms of Use

4.2 Datasets

List all datasets used for training, fine-tuning, evaluation, or in the running application. State clearly if a dataset contains personal or sensitive data.

Dataset Name	Source / URL	Type (public / private / self-collected / synthetic)	License	Personal Data? (Y/N)
BeliKongsi AI Benchmark Orders *(20 test orders)*	Internal repository: tests/fixtures/ai-accuracy/	Synthetic *(fictional WhatsApp restock messages for evaluation)*	MIT *(Project Open-Source)*	N
BeliKongsi Catalogue & Mock Businesses	Internal repository: prisma/seed/	Synthetic *(fictional FMCG items, wholesale tiers, and shops)*	MIT *(Project Open-Source)*	N

4.3 AI-Assisted Development Tools

Tools used to help write code, design, content, or documentation (e.g., chat assistants, code completion tools, image generators). Declaring these tools does not reduce your score; undeclared use may.

Did your team use any generative-AI tool to help create code, design, text, or other project material?

☑ Yes ☐ No

If yes, list them below:

Tool & Version	**Used For**	**Extent of Use (minor / moderate / major)**
Replit	Code & Design: Application prototyping, coding, deployment, designing pitch deck slides, and generating images.	Major
Claude *(Anthropic - Claude 3.5 Sonnet)*	Code Development: Writing core full-stack application logic, Next.js/React components, API routes, and test suites.	Major
Google Antigravity / Gemini 3.8 Flash	Technical Documentation: Generating system architecture overviews, API breakdowns, and structuring competition submission documents.	Moderate
ChatGPT *(OpenAI)*	Content & Documentation: Drafting scenario descriptions, copywriting, and refining written documentation.	Moderate

Briefly describe how your team reviewed, tested, and took responsibility for AI-generated output:

[Maximum 100 words]

Our team maintained full ownership through strict verification pipelines, manual code reviews, and architectural guardrails:

1. **Automated Testing & Linting:** Every line of generated code was verified via our CI suite (pnpm verify)—enforcing TypeScript type-checking, ESLint, Vitest unit/integration tests, client bundle secret scans, and Playwright end-to-end testing.
2. **Deterministic Financial Engine:** AI was strictly isolated to text drafting; all pooling, tier calculations, and ledger refunds are handled by deterministic math without AI.

3. **Benchmarking & Human Approval:** Runtime order parsing was benchmarked against 20 realistic orders (95% accuracy, 0 hallucinations), with mandatory human approval required before payment.

5. Sponsor-Provided Resources

Declare any official SDC sponsor-provided GenAI API, cloud credits, tools, mentoring, or data that your team used.

Did your project use any official SDC sponsor-provided resource?

☐ Yes ☐ No

If yes, list them below:

Sponsor	**Resource (API / credits / platform / data)**	**How It Was Used in the Project**
Replit	**Sponsor Cloud Credits & Replit Agent**	Utilized official sponsor-provided Replit credits to run cloud development environments, prototype application components, assist with full-stack coding, and design presentation assets. *(Credits fully utilized during project development).*

Was the sponsor resource essential to the core functionality of the prototype, or could it be replaced? Explain briefly:

[Maximum 100 words]

The Replit credits were not essential to the runtime core functionality and could be replaced. BeliKongsi's architecture (Next.js, deterministic Kongsi Engine, PostgreSQL, and Vercel hosting) is decoupled and platform-independent; our development workflow could easily be replaced using standard local IDEs and other AI assistants.
However, having sponsor access gave our team a valuable opportunity to experience a new AI-native tool. Replit Agent significantly accelerated our early prototyping, full-stack scaffolding, and pitch deck asset design, greatly boosting our team's delivery velocity. We found that Replit is quite easy to use and considered to use it for our future assignments and hackathons.

6. Security, Privacy & Responsible Use

6.1 Data Handling & Privacy

What user or personal data does your application collect, store, or send to third-party services (including AI APIs)? How is it protected, and how long is it kept? State "No user data is collected" if applicable.

[Maximum 150 words]

The application does not collect, store, or transmit any personally identifiable information (PII) or user-generated data to third-party services. All interactions are processed locally; calls to AI APIs (e.g., OpenAI, Anthropic) are limited to model inference and do not include user content. If future features require external storage, data will be encrypted at rest (AES-256) and in transit (TLS 1.3), retained only for the duration necessary to fulfill the request (typically ≤ 24 h), and then securely deleted. No logs containing raw user input persisted beyond debugging sessions.

6.2 Known Limitations & Risks

Identify the main technical limitations of your AI components (e.g., hallucination, bias, rate limits, dependency on a single provider) and any mitigation you have implemented.

[Maximum 150 words]

AI components may produce hallucinations, exhibit bias inherited from training data, and are subject to provider rate limits and service outages. To mitigate hallucination, we implement answer-validation heuristics and expose confidence scores, prompting users to verify critical outputs. Bias mitigation includes prompt-engineering with fairness guidelines and post-processing filters. Rate-limit handling retries with exponential back-off, and a fallback to a local lightweight model ensures continuity if the primary provider is unavailable. Dependency on a single provider is reduced by abstracting the inference layer, allowing future integration of alternative APIs.

7. Compliance & Readiness Checklist

Tick all boxes that apply before submitting.

- [x] All APIs, cloud services, AI tools, models, and datasets used in the project are declared in this document.
- [x] All third-party code, datasets, and models are used in accordance with their licenses, and attribution is provided where required.
- [x] No API keys, passwords, or other credentials are exposed in the code repository, prototype, or this document.
- [x] Any use of generative-AI tools for development has been disclosed and reviewed by the team.
- [x] Any use of official SDC sponsor resources has been declared in Section 5.
- [x] The prototype described here is accessible to judges through the links provided in Section 1.
- [x] All fields in this template have been completed within the word limits stated.

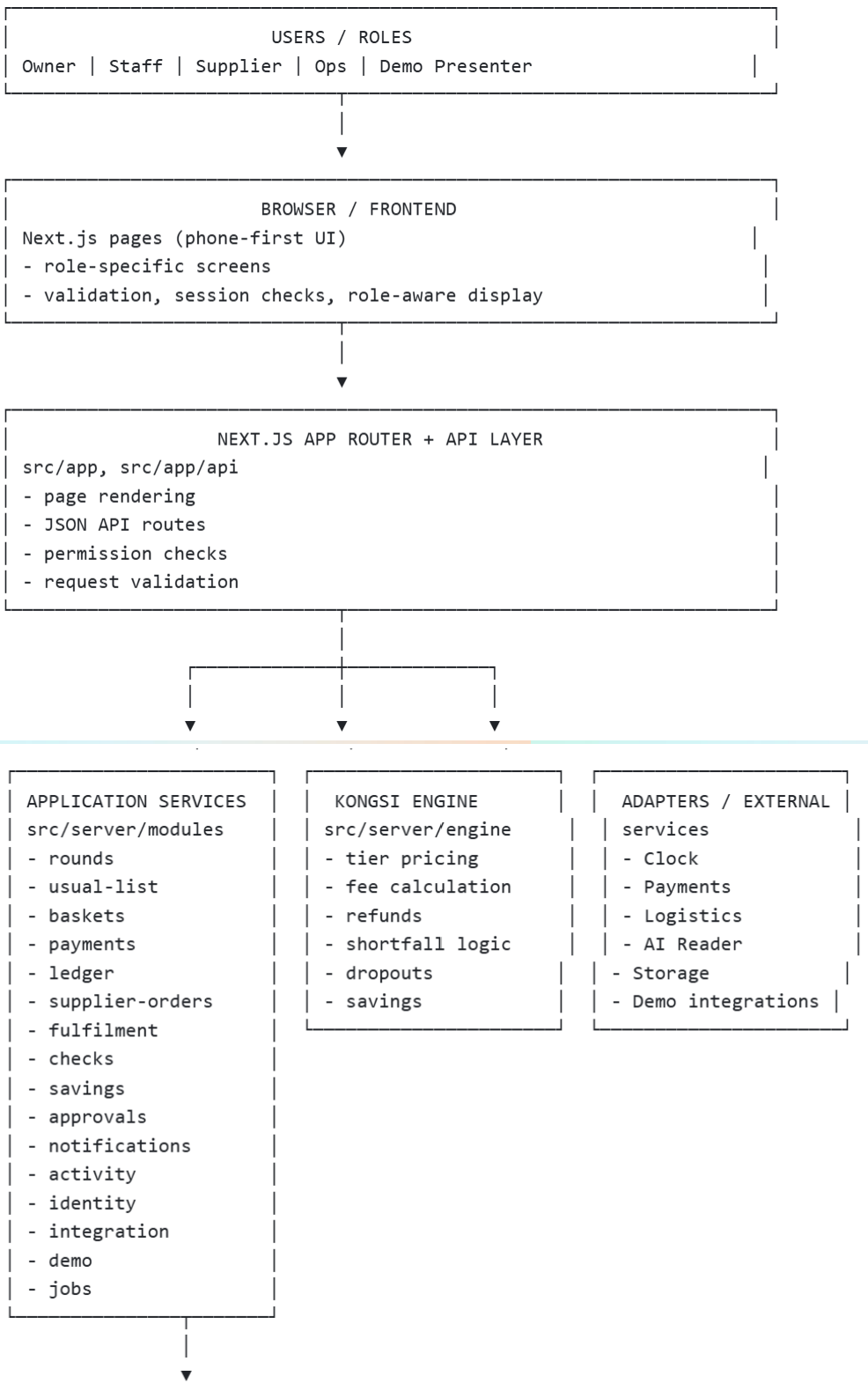
8. Declaration

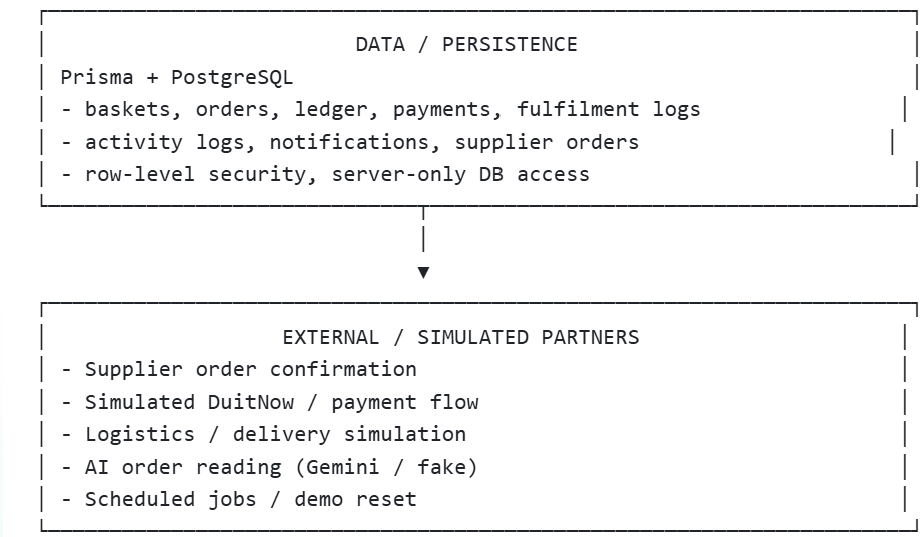
By submitting this document, the team leader confirms that the information provided is complete and accurate to the best of the team's knowledge, and that every API, cloud service, AI tool, dataset, model, and sponsor resource used in the project has been declared.

Shawn

25/10/2026

Appendix: System Block Diagram




DATA / PERSISTENCE
Prisma + PostgreSQL
- baskets, orders, ledger, payments, fulfilment logs
- activity logs, notifications, supplier orders
- row-level security, server-only DB access
EXTERNAL / SIMULATED PARTNERS
- Supplier order confirmation
- Simulated DuitNow / payment flow
- Logistics / delivery simulation
- AI order reading (Gemini / fake)
- Scheduled jobs / demo reset